

Name: sf-pet-care team

Team Members

Ahmad Naggayev	Engineer
Zhidan Niu	Engineer
Maurice Onyonyi	Engineer
Jonah Robinson	Engineer
Thomas Sasser	Engineering Manager

Git URL: <https://github.com/Morloh/sf-pet-care>

GCP Project: <https://console.cloud.google.com/welcome/new?authuser=1&project=sf-pet-care>

Work Plan

Each person has been assigned tasks in our Slack sf-pet-care Project Tracker.

<https://usfca.enterprise.slack.com/lists/T0GQ64ARG/F0C4L6E6GRM>

Ahmad Naggayev:	ETL Integration on 'ahmad' branch
Zhidan Niu:	Community Survey, documentation
Maurice Onyonyi:	Business Location
Jonah Robinson:	Neighborhood Analysis
Thomas Sasser:	Sprint, meetings, QA, integration, PR, documentation

Sprint Plan

A sprint has been created in Slack called "sf-pet-car Project Tracker". Each team member has assigned tasks, including:

Ahmad Naggayev:	Create Unified ETL Pipeline, Research on how to do ETL
Zhidan Niu:	Create Community Survey Script, Overview Documentation
Maurice Onyonyi:	Create Business Locations Script
Jonah Robinson:	Create SF Neighborhood Analysis Script, Create GCP Project
Thomas Sasser:	Scrum Master, Work Contract, Program Management, QA, integration, PR and documentation.

Everyone will participate in code reviews and PR meetings.

Meeting Schedule

We will have daily standup first, with a post-standup meeting for clarifications. We will also have daily work sessions to ensure everyone is on track, get help where needed, etc. Times are variable, but for the weekend:

Saturday 10/3/26:

Standup: 1600

Work Session 1: 1615

Work Session 2: 2100

Sunday 10/4/26:

Standup: TBD

Work Session 1: TBD

General Work Contract

We are using Slack's Project Tracker and our instance is called "sf-pet-care project tracker" under the "Files" section in Slack.

Tasks will be clearly stated and assigned. Every team member has the power to update a task if additional requirements are discovered or clarifications are necessary. Changes should be discussed with the team unless the change is obvious and should be documented. Team members are encouraged to create additional tasks if they spend a significant amount of time on a problem that is related to the task. This documents the problem and perhaps may help another engineer.

Status for tasks include: "Not Started", "In Progress", "Blocked" and "Done".

Each team member is responsible for closing their tasks. This is the last step and occurs after a PR is merged.

Phase 2's project deadline is Sunday, October 4. Everything needs to be completed prior to that so the team has time to fix bugs and finalize all details. The goal is 11:59 PM on Saturday, October 3.

Project Branches

The general pipeline for code flow is from a personal branch to dev to main.

Each team member will create a personal branch to use for development. This should be the engineer's first name. The "dev" branch will be used to test all code. Once dev passes the team's tests and meets the project requirements, the team shall have a meeting to approve the final PR into the main branch.

Each merge requires a PR. The conditions for which a task is eligible for is as follows:

- 1) Code is created in the personal branch.
- 2) The code passes the engineer's tests and can demonstrate that the code works as expected.
- 3) Any documentation is complete.

GCP

Engineers will use the GCP project 'sf-pet-care'. All buckets will have "-dev" and "-prod" tags appended to the name. "-dev" is to be used for code creation and the data can be changed, deleted, etc. "-prod" data should never be deleted.

Scripts

Engineers will create scripts for each data source on your individual branch according to the requirements in the tasks. Scripts shall have

a dev flag set to True or False. This will change the config for the script. If dev=True, then the "-dev" bucket will be used. Conversely, if dev=False, then the "-prod" bucket will be used.

Scripts shall be named <team_name>_<what_it_is>.py. This removes ambiguity.
Example: sf-pet-care-neighborhood-analysis.py.

Pull Requests

A PR is required for any script to move outside its originating branch. The intent is for the entire team to review the code, comment on it, and approve or reject the PR. Documentation must accompany the code in order for it to be eligible for a PR. Additionally, the author shall be able to demonstrate that the code meets the requirements. Comments do not need to be lengthy, but they do need to be relevant.

Once we have declared the project complete, we will have a final meeting to review the PR that will merge into main. This will then be submitted for grading.